

ITF23019: Parallel and Distributed Programming

Spring Semester, 2026

Lab8: Additional Thread-Safety Issues

Submission Deadline: March 11st, 2026 23:59

You need at least 50 pts to pass this lab assignment.

In this lab, we look beyond the basic synchronization mechanisms and explore additional issues of Thread safety in a multithread program. These issues often stem from deeper aspects of the Java Memory Model, the interaction between threads, and the behavior of shared data under concurrency. Topics include the use of atomic variables to guarantee indivisible operations, the role of the volatile keyword in ensuring memory visibility among threads, and the consequences of holding locks for too long. We will also study how deadlocks occur, how to detect the deadlock and how to fix deadlock problem.

1 Using atomic variables instead of synchronization

In theory, when a CPU executes one instruction, the execution is divided into different stages depending on processors. For example, a Reduced Instruction Set Computer (RISC) processor typically has five stages. In many cases, execution can be grouped into three main stages: the CPU first fetches data or instructions from the main memory (RAM), then decodes and executes the instructions, and finally writes the data back to memory. In a multi-threaded program, different stages of an instruction can overlap with stages of other instructions during execution.

To avoid the data inconsistency problem caused by overlapping instruction executions, an **atomic action** in which a group of related operations are either all performed without interruption or not performed at all can be used. Atomic actions are important to avoid race conditions. One explanation for this is illustrated in Figure 1. As can be seen in the figure, when all stages on instruction `count++` or `count--` are performed together, we have expected output: `count` equals to 0.

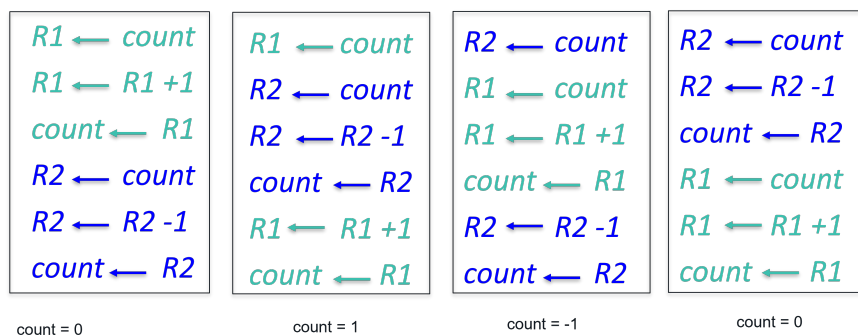


Figure 1: Different output of a program with different order of instruction executions.

Since version 5, Java includes atomic variables to provide atomic actions on single variables. When a thread is doing an operation with an atomic variable, the implementation of the class includes a mechanism to check that the operation is done in one step. It is similar to that we can 'glue' all related stages together into one operation and they cannot be intervened by executions of other threads in the program. The most important atomic variables which can be used in Java are:

- AtomicInteger
- AtomicLong
- AtomicReference
- AtomicBoolean
- LongAdder
- DoubleAdder

Be aware that in some cases, atomic variables offer a better performance than solutions based on synchronization mechanisms (specially when we care about atomicity within each separate variable).

Project `atomicVariableExample` demonstrates how to solve a data race problem using atomic variable. In the package `dataRaceExample` (same as in lab6), multiple threads are executed without any synchronization mechanism. The project creates two threads, both of which can access a shared variable `count`. One thread increases `count` by 10 multiple times, while the other thread decreases `count` by 10 the same number of times. Therefore, the expected output should be 0. However, the actual output varies significantly with each run. The code below fixes the problem by using atomic variable (in the package `dataRaceFixed`).

```
import java.util.concurrent.atomic.AtomicInteger;
class SharedCounter {
    static AtomicInteger count = new AtomicInteger(0);
}
class IncreaseThread extends Thread {
    public void run() {
        for (int i = 0; i < 1000000; i++) {
            SharedCounter.count.addAndGet(10);
        }
    }
}

class DecreaseThread extends Thread {
    public void run() {
        for (int i = 0; i < 1000000; i++) {
            SharedCounter.count.addAndGet(-10);
        }
    }
}

public class DataRaceFixed {
    public static void main(String[] args) {
        Thread t1 = new IncreaseThread();
        Thread t2 = new DecreaseThread();

        t1.start();
```

```

        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Final count: " + SharedCounter.count);
    }
}

```

Here, we declared an `AtomicInteger` variable named `count` to store the value of `count` after each thread execution. The `count` is protected. When we execute the program, we will see that the final value of the variable `count` remains the same as at different runs with `Final count: 0`.

2 Volatile Keyword

A potential problem with shared-memory systems is cache coherence. The same variable can be stored in the caches of two different cores, and if one core updates the value of the variable, the other core may be unaware of the change. Under some circumstances, you can use the `volatile` keyword and not use a synchronization mechanism. If only one of the tasks modifies the data and the rest of the tasks read it, you can use the `volatile` keyword without any synchronization or data inconsistency problems. In other scenarios, you need to use a lock, the `synchronized` keyword, or any other synchronization method.

`Volatile` keyword in Java can be used to solve CPU cache coherence problem. `Volatile` keyword is a modifier that allows programmers to specify that a variable must always be read from and stored in the main memory, not in a CPU cache. `Volatile` keyword is used when the visibility of the actual values is important, not the order of thread execution. If the order of thread execution is important, a synchronization mechanism has to be used.

The project `volatileExample` is one example using the `Volatile` keyword to avoid data inconsistency problem caused by Cache coherence. The project has two threads: a normal thread, executes the task defined in `Task` class and a `Volatile` thread, executes the task defined in `VolatileTask` class. For the normal thread, the public attribute `flag`, declared in the `Flag` class, is initialized to the `true` value. As long as this attribute is `true`, the thread continuously increases `i` attribute, as shown in the below code snippet.

```

public void run() {
    int i = 0;
    while (flag.flag) {
        i++;
    }
    System.out.println("Never be there");
    System.out.printf("Task: Stopped at count %d - %s\n", i, new Date());
}

```

The `main()` in the `Main` class tries to stop the execution of the normal thread by changing the `flag` to `false`, shown in the below code snippet.

```

System.out.println("Change Flag in Task to stop its execution");
flag.flag=false;

```

```
System.out.println("Main: The Remaning Thread stops ?");
```

The logic is the same for the Volatile thread. The public attribute `flag`, declared in `VolatileFlag` class, is also initilized to the `true` value. However, unlike the `flag` in the normal thread, we add the `volatile` modifier to the declaration of the `flag`.

```
public volatile boolean flag=true;
```

Similar to the normal thread, the Volatile thread continues increasing the attribute `i` as long as `flag`, declared in `VolatileFlag` class, remains `true`.

```
public void run() {
    int i = 0;
    while (flag.flag) {
        i++;
    }
    System.out.printf("Volatile Task: Stopped at count %d - %s\n", i, new Date());
}
}
```

The `main()` in the `Main` class tries to stop the execution of the Volatile thread by changing the public attribute `flag` to `false`, shown in the below code snippet.

```
System.out.println("Change Flag in Volatile Task to stop its execution");
volatileFlag.flag=false;
System.out.println("Main: Volatile Thread stops ");
```

The following is the output of running the project

```
Change Flag in Volatile Task to stop its execution
Main: Volatile Thread stops
Volatile Task: Stopped at count 10699701 - Wed Mar 04 17:08:43 CET 2026
Change Flag in Task to stop its execution
Main: The Remaining Thread stops ?
```

The output shows that the application never stops because the normal thread never finishes its execution. This can be explained as follows. When the `main()` changes the value of `flag` in the Volatile thread, the new values is written in the RAM. This signals the Volatile thread, which executes the task defined in `VolatileTask` to stop increasing `i`, print two more messages and then finishes. On the other hand, when the `main()` modifies the value of `flag` in the normal thread, the new value is written in the cache but this new value is not updated in RAM. The thread which executes task defined in the `Task` class does not see the updated value, as a result, it is in the infinitive loop and its execution will never end.

3 Holding locks for as short a time as possible

Locks, like any other synchronization mechanism, allow you to define a critical section that only one task can execute at a time. While a task is executing the critical section, the other tasks that want to execute it are blocked and have to wait for the liberation of the critical section. The application is working in a sequential way. You have to pay special attention to the instructions you include in your critical sections because you can degrade the performance of your application without realizing it. You must make the critical section as small as possible, and it must include only the instructions that work on shared data with other tasks, so the time that the application is executing in a sequential way would be minimal.

The project `criticalSection` is one example of holding a lock. The `Operations` class define three methods which can be accessed by different threads. (For simplicity of implementation, these methods only include `sleep` functions). Supposed that only `processData()` and `writeData()` need mutual exclusion. The project illustrates two different approaches for implementing the critical section. The first approach is to put all methods in the critical section and the other is to put only methods which needs mutual exclusion in a critical section, shown in the code snippet below.

```
public void run() {
    lock.lock();
    try {
        Operations.readData();
        Operations.processData();
        Operations.writeData();
    } finally {
        lock.unlock();
    }
}
```

and

```
public void run() {
    lock.lock();
    try {
        Operations.processData();
    } finally {
        lock.unlock();
    }

    Operations.readData();

    lock.lock();
    try {
        Operations.writeData();
    } finally {
        lock.unlock();
    }
}
```

When running the program, it can be seen that the execution times are different among the two approaches. The output of program shows the different executions time:

```
Main: First Approach: 300507
Main: Second Approach: 300199
```

4 Deadlock

4.1 (Short) Introduction

In theory, a set of processes is said to be deadlocked if each process is waiting for an event that only another process can cause, as can be seen in Figure 1 and Figure 2. When deadlock happens, all processes in the set are waiting and none of them causes any event that could wake up any of process. They all wait forever. None of them can run, none of them can release any resources, and none of them can be

awakened.

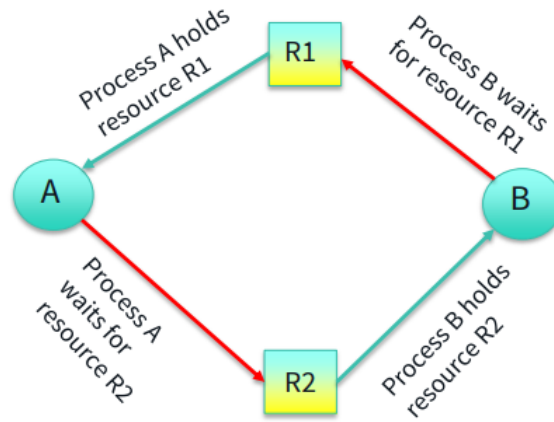


Figure 2: Deadlock with two processes in a set.

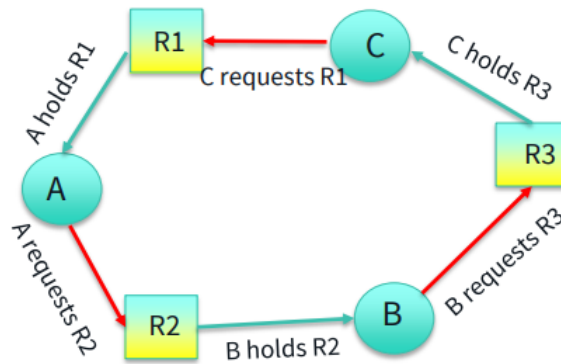


Figure 3: Deadlock with three processes in a set.

4.2 Deadlock in a multithread program in Java

In a multithread program where multiple threads can access to a shared resource, deadlock can occur if **synchronized** is not used properly. In this lab assignment, you will see how deadlock will happen when using **Monitor** and **Lock**.

In this scenario, two or more threads are waiting for resources held by each other to proceed their execution, resulting in a situation where none of the threads can proceed. Deadlock in Java can happen if the **synchronized** is not used properly.

In the code snippet below (in the project **deadlock**), two threads are created in a program. Each of which is trying to acquire two different resources in a different order. Thread1 is holding resource R1 and is waiting for resource R2. On the other hand, Thread2 is holding resource R2 and is waiting for resource R1. If both threads run simultaneously, they may acquire one resource each, and then wait indefinitely for the other thread to release the resource they need. This results in a deadlock where none of them can make any progress.

```

private static class Thread1 extends Thread {
    public void run() {
        synchronized (resource1) {
            System.out.println("Thread 1: Holds resource R1...");
            try {
                TimeUnit.SECONDS.sleep(5);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
            System.out.println("Thread 1: Waits for resrouce R2...");
            synchronized (resource2) {
                System.out.println("Thread 1: Holds resource R1 and R2...");
            }
        }
    }
}

private static class Thread2 extends Thread {
    public void run() {
        synchronized (resource2) {
            System.out.println("Thread 2: Holds resource R2...");
            try {
                TimeUnit.SECONDS.sleep(5);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
            System.out.println("Thread 2: Waits for resource R1...");
            synchronized (resource1) {
                System.out.println("Thread 2: Holds resource R1 and R2...");
            }
        }
    }
}

```

Output of running the program:

```

Thread 1: Holds resource R1...
Thread 2: Holds resource R2...
Thread 2: Waits for resource R1...
Thread 1: Waits for resrouce R2...

```

Note that deadlocks can occur in a variety of situation and can hard to detect and diagnose. The first sign of the deadlock is that your program keeps running.

5 Exercises

5.1 Exercise 1 (30 pts)

The project `dataRace` implements 10 increasing tasks and 10 decreasing tasks. Each increasing task increases a shared variable `count` by 1, while each decreasing tasks decreases the variable `count` by 1. The expected output of `count` is 0. However, the actual output of `count` varies with each run and is never 0.

- Use `atomic variable` to fix the data race problem.

Note: The project `dataRace` is the correct version of the project `raceCondition` in lab6!

5.2 Exercise 2 (20 pts)

- Run the code in package `volatileExample` and include the output in your report.
- How long does it take for the program to finish running?

5.3 Exercise 3 (20 pts)

- Run the code in package `criticalSection` and include the output in your report.
- Explain why execution times are different between two approaches.

5.4 Exercise 3 (30 pts)

Refer to the project `deadlock`. Fix deadlock problem in `deadlockMonitor` class and in `deadlockLock` class.

6 What To Submit

Complete the the exercises in this assignment. Put your codes along with your `lab8_report` file (Markdown or pdf file) inside the `lab8` directory of your repository.

7 References

1. González, Javier Fernández. Mastering Concurrency Programming with Java 9. Packt Publishing Ltd, 2017.
2. Class textbook.